From Perceptron to Deep Networks: Why Hidden Layers Matter

Student ID: 24152801
Module Name: Machine Learning and Neural Networks
Github Repository: https://github.com/Bhavesh2618/Machine-Learning-Tutorial-24152801

Abstract:

The key element to modern machine learning is the artificial neural networks, which are more effective because of the layer structure than the single neurons. This tutorial discusses how the perceptron was improved to become deep neural networks in which hidden layers play an important role. A perceptron is a simple linear classifier which calculates a weighted average of the inputs and uses a threshold. Although it works well in cases of linearly separable data it is not able to generalize complicated relationships. This can be bypassed with hidden layers which allow non-linear mapping and learned hierarchies of features.

The study compares three types of models such as a perceptron-like model, a shallow multilayer perceptron (MLP), and a deeper MLP on the Breast Cancer dataset. As it turns out, representational capacity does get larger with the addition of hidden layers, but this does not necessarily improve performance. This shows that the complexity of models should be with characteristics of datasets. The tutorial concludes that non-linear learning requires hidden layers though the ideal number of hidden layers relies on the problem.

Introduction:

Machine learning allows systems to acquire the data patterns rather than through direct programming. One of the most effective approaches in the field, and based on biological neurons, is neural networks. These networks are interrelated layers which process data on the basis of weighted transformations.

The simplest model of the neural network is the perceptron. It calculates a weighted average of the inputs and uses a threshold that produces an output. Although it is helpful, it is restricted to linear decision boundaries and is unable to tackle non-linearly separable issues.

Neural networks do this to address this limitation by adding hidden layers. The layers change the input data into something more meaningful then the data is classified. This development of perceptron to deep networks is critical in the study of the modern artificial intelligence systems.

Objective:

This tutorial aims at examining the use of hidden layers in neural networks. Specifically, it aims to:

- Compare perceptron, shallow and deep neural networks.
- Test them with a real-world set of data.
- Learn depth and its influence on learning and generalisation.

Background Theory:

The perceptron is defined as:

$$\boldsymbol{z = w_1 x_1 + w_2 x_2 + \cdots + w_n x_n + b}$$

where weights are learned from data and $b$is the bias.

This is expanded by a multilayer perceptron that adds layers of the hidden layer. Every layer implements a transformation with an activation function, e.g. ReLU. This allows the network to acquire non-linear relations.

The neural networks can use the hidden layers to:

- Get to know complex interaction of features.
- Change data to higher levels of representation.
- Unprecise non-linear functions.

Deep networks have several hidden layers making them more expressive.

Methodology:

Breast Cancer scikit-learn dataset is utilized. It contains:

- 569 samples
- 30 numerical features
- Binary (malignant vs benign).

Steps performed:

- Train-test split (80/20)
- StandardScaler feature scaling.
- Training for 50 epochs

There were three models that were applied:

- No hidden layers Perceptron-like model.
- The model has a shallow MLP (1 hidden layer and 16 neurons).

- Deep MLP (two layers with 32 and 16 neurons).

Results:

Final test accuracy:

- Perceptron-like model: 0.9649
- Shallow MLP: 0.9649
- Deep MLP: 0.9561

Figures:

Validation Accuracy:

The perceptron like and shallow MLP models are almost identical in terms of validation accuracy which was about 0.9649. It suggests that the dataset can be separated linearly to a great extent, and even a simple model can work. The deeper MLP has a little less validation accuracy which indicates that further depth does not enhance generalisation.

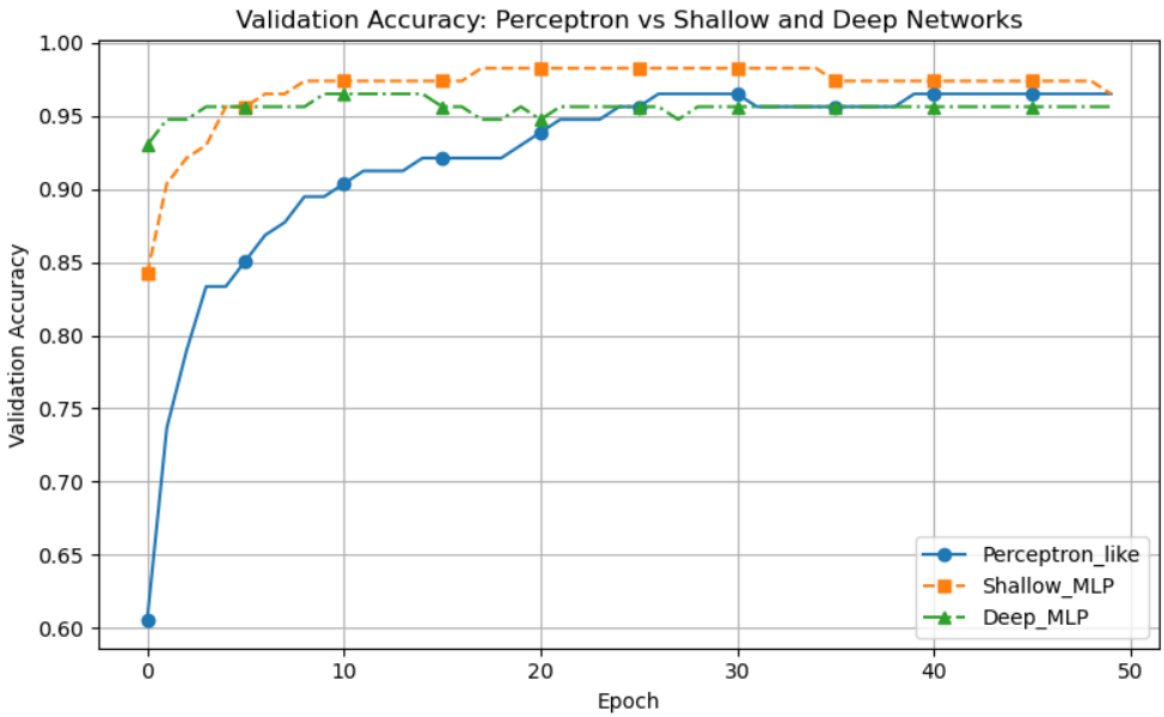

Figure 1: Validation accuracy across training epochs

Training Loss:

The loss also decreases with time; which clearly shows that learning took place. The deeper model has less training loss which implies that it is closer to training data. Nonetheless, this does not lead to better validation performance meaning that there is a possibility of overfitting.

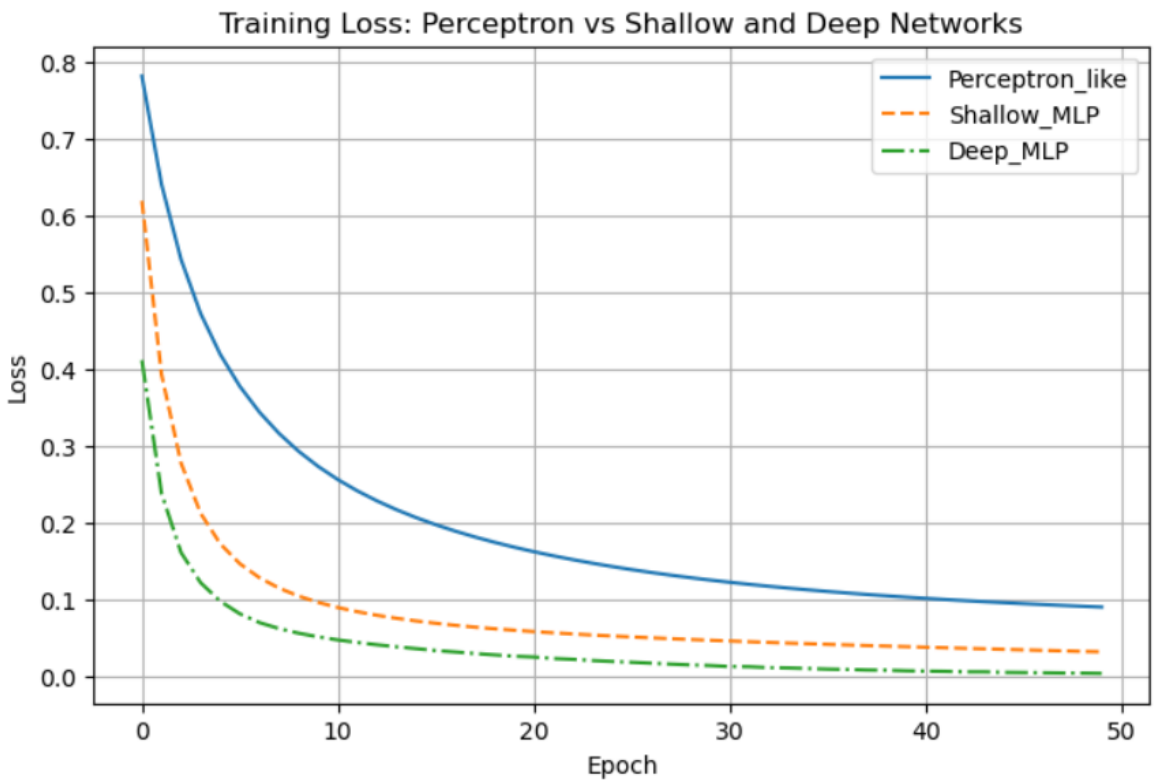

Figure 2: Training loss curves for perceptron-like, shallow, and deep neural networks.

Final Accuracy Comparison:

The perceptron-based and shallow MLP models are the same in terms of the accuracy (0.9649), whereas the deeper model has a marginally lower accuracy (0.9561). This is a case of diminishing returns to depth.

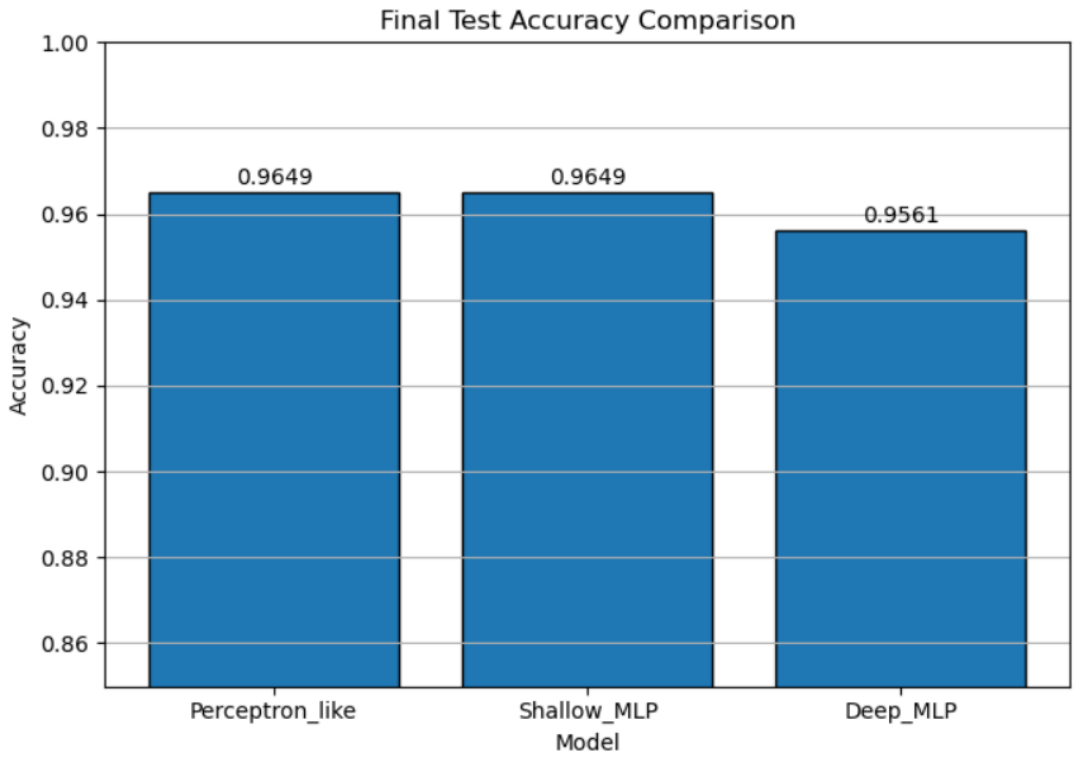

Figure 3: Final test accuracy comparison between models.

Confusion Matrix Analysis:

The confusion matrices indicate that all the models are able to classify most of the samples correctly with a few misclassification occurring. The more profound model presents a few additional errors, which confirms the fact of lower generalisation performance.

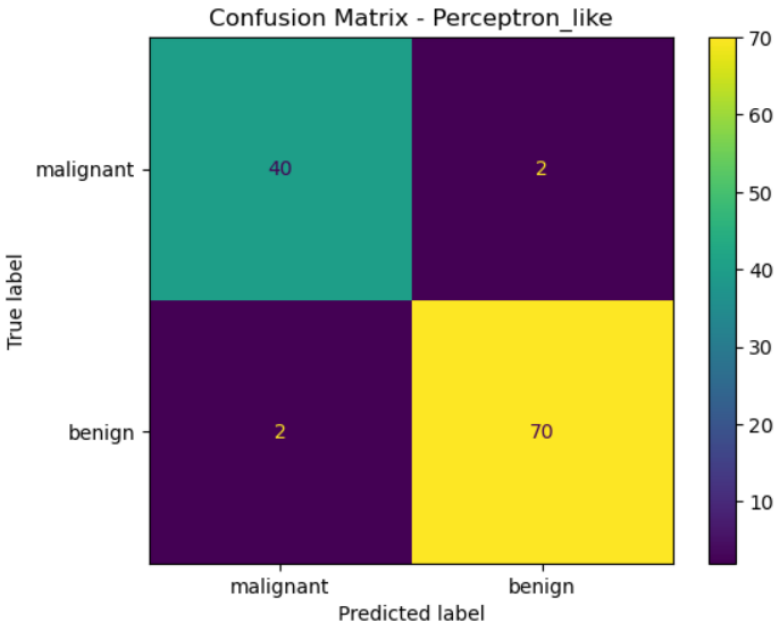

Figure 4: Confusion matrix for perceptron_like

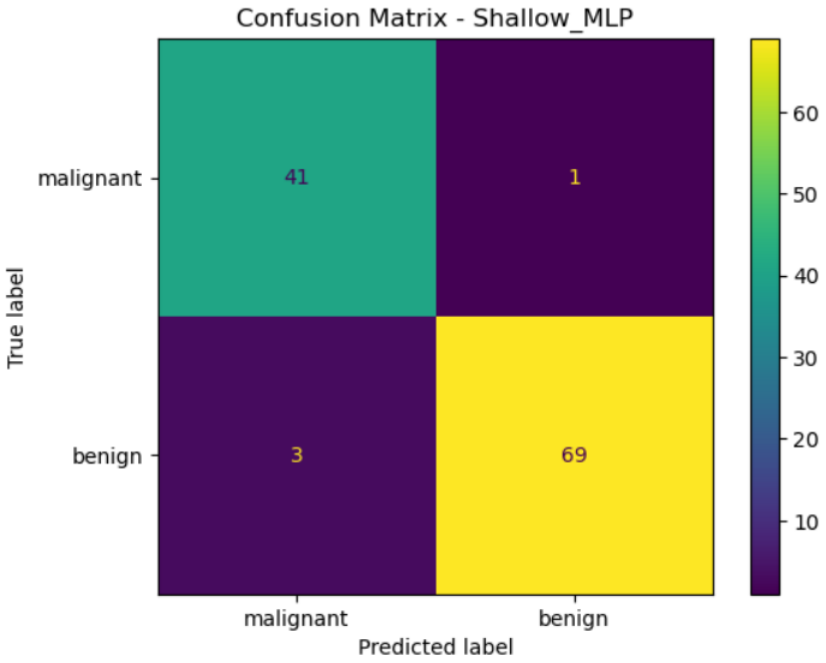

Figure 5: Confusion matrix for Shallow_MLP

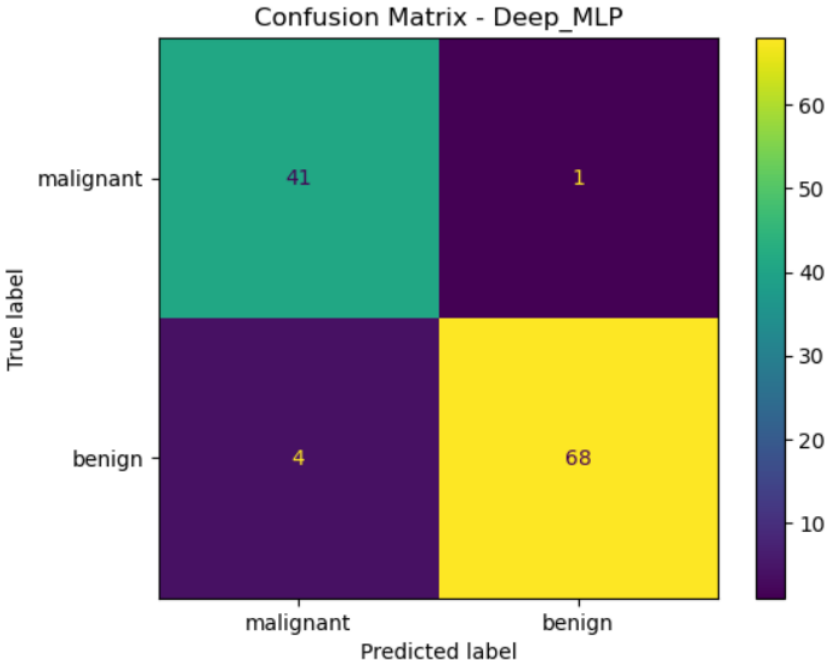

Figure 6: Confusion matrix for Deep_MLP

Interpretation of Results:

The shallow MLP models, perceptron-like, and both models were in the same performance, which means that the dataset is mostly linearly separable. The higher MLP obtained marginally lower accuracy implying that further complexity of the model might inject overfitting without further enhancing generalisation.

This shows that getting deeper does not necessarily mean achieving better performance.

Discussion:

These results support the fact that the complexity of models should be equal to the complexity of data. Although, the expressive power of hidden layers is enhanced, it remains only useful in cases where non-linear transformations of the underlying data are necessary. Extra layers can add redundant parameters in simple data sets, and this will cause overfitting and decreased efficiency of the model. This shows that the neural network depth must be viewed as a design decision but not as an implicit optimisation approach.

Accessibility Considerations:

This tutorial has been done with accessibility in picture.

- There are clear axis labels, legends, and grid lines of visualisations to enhance readability.
- The models are differentiated by using different line styles (solid, dashed and dotted) and markers which make plots not entirely based on colour differences.
- Bar charts also have numeric labels to enable a specific interpretation as opposed to visual estimation.
- Contrast colour schemes are applied to enhance visibility.
- Numbers have descriptive notes about important insights.
- The report layout allows the screen reader navigation due to clear headings and logical organisation.

The accessibility of colour blindness has been made special consideration. Visualisations are created in such a way that one can appreciate the difference between models and not just recognise it by colour. This makes sure that all the plots can be interpreted by the users who have a colour deficiency vision.

Ethical Considerations:

Machine learning in this case can give biased results when the relevant data is biased. This may result in unjust practices in such sensitive areas like healthcare. Besides, deeper neural networks decrease interpretability. The models that are simpler are easy to comprehend unlike the complex models which are black boxes. Thus, there should be a compromise between transparency and accuracy.

Conclusion:

This tutorial shows that the hidden layers are necessary in facilitating the non-linear learning in neural networks. As much as they enhance model flexibility, they are not necessarily performance enhancers.

The findings demonstrate that less complex models can be used on structured data, and more complex networks are applicable on complex problems. Thus, the layers that are not visible are important, although the depth is best determined by the data.

References:

1) Goodfellow, I., Bengio, Y. and Courville, A. (2016) *Deep Learning*. MIT Press.
2) Pedregosa, F. et al. (2011) 'Scikit-learn: Machine Learning in Python', *Journal of Machine Learning Research*, 12, pp. 2825–2830.
3) LeCun, Y., Bengio, Y. and Hinton, G. (2015) 'Deep learning', *Nature*, 521(7553), pp. 436–444.
4) Bishop, C.M. (2006) *Pattern Recognition and Machine Learning*. Springer.
5) Mitchell, T.M. (1997) *Machine Learning*. McGraw-Hill.
6) He, K., Zhang, X., Ren, S. and Sun, J. (2016) 'Deep residual learning for image recognition', *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp. 770–778.